MODULE 18 · DOMAIN 4 - PERFORMANCE OPTIMIZATION, QUERYING, AND TRANSFORMATION

The three caches, and why a query is slow

Three caches in two layers, and a decision tree for the commonest question in this domain.

DOMAIN 4 of 5 WEIGHT 21% MODULE 18 of 31

admin@sthithapragna.com

Two caches live in **cloud services**. One lives **on the warehouse**. That single fact sorts almost everything.

The three caches

CACHE	WHERE IT LIVES	WHAT IT HOLDS
Result cache	Cloud services	The complete result of a previous query.
Metadata cache	Cloud services	Micro-partition statistics: row counts, value ranges, distinct counts.
Warehouse local disk	On the warehouse	Micro-partition data recently read by that warehouse, on local SSD.

ON THE EXAM

Only the third needs a running warehouse. Only the third is lost when the warehouse suspends.

The result cache: how long it lasts

- A result persists for **24 hours from its last use**. Every reuse **resets** that 24-hour window.
- There is an absolute ceiling of **31 days from first execution**. After that it is purged and regenerated.
- Reuse consumes **no warehouse compute at all**, because the result comes from cloud services.
- Controlled by **USE_CACHED_RESULT**, settable at **account, user or session** level. Enabled by default.

> ON THE EXAM
>
> Candidates remember the 24 hours and drop the 31-day ceiling. A harder question targets exactly that gap.

The result cache: when it will not be reused

- **The query text must match exactly.** A change of case, a table alias, or extra whitespace prevents reuse.
- **The data must be unchanged.** Any DML on a referenced table invalidates it.
- **The micro-partitions must be unchanged**, so **reclustering also invalidates it**, even with no DML.
- The role must hold the **required privileges on every table** involved.
- No functions evaluated at execution time, such as **CURRENT_TIMESTAMP or RANDOM**. No UDFs or external functions.

> ON THE EXAM
>
> Five ways to miss the cache, and the first is the one people never suspect. Formatting a query differently is enough.

The other use of persisted results

- **RESULT_SCAN** queries the result of a previous statement as if it were a table, using its query ID.
- **RESULT_SCAN(LAST_QUERY_ID())** is the common form. Build a query step by step without recomputing earlier stages.
- It is the standard way to filter or join the output of **SHOW**, **DESCRIBE** and **CALL**, which return results that are otherwise hard to reuse.
- It reads the **same persisted results** as the result cache, so the same 24-hour window applies.

ON THE EXAM

Retrieval optimisation and post-processing are two uses of one mechanism. RESULT_SCAN is the second.

The metadata cache

- Answers from **micro-partition statistics** rather than from data. Row counts, value ranges, distinct counts.
- **SELECT COUNT(*)** on a large table can return instantly with **no warehouse running** and no credits consumed.
- Some MIN and MAX queries are answered the same way, where the recorded statistics are sufficient.
- This is the same metadata that drives **pruning** in module 3. One store, two uses.

> ON THE EXAM
>
> If a scenario says a query on a huge table returned in milliseconds with nothing running, this is usually the answer.

The warehouse local disk cache

- Holds micro-partition data that **this warehouse** has recently read, on its local SSD.
- **Dropped when the warehouse suspends.** The next queries run cold and re-read from storage.
- **Specific to one warehouse.** Another warehouse reading the same table builds its own cache from scratch.
- **Larger warehouses have more local storage**, so they hold more of the working set.

> ON THE EXAM
>
> This is the other half of the auto-suspend trade from module 17. Suspending saves credits and throws away the cache.

Why is this query slow?

01

Queued?

Waiting for compute. Scale out, or reduce concurrency.

02

Scanning too much?

Partitions scanned near total. Clustering, or a better filter.

03

Spilling?

Bytes spilled local or remote. Scale up for more memory.

04

Exploding join?

Rows out far exceed rows in. Fix the join condition.

THE DECISION TREE

Four questions in order. Each has a different remedy, and the Query Profile answers all four. Module 19 reads them.

How this module gets tested

THE OBSERVATION	THE EXPLANATION
Instant result, no warehouse running, no credits	Result cache, or metadata cache for COUNT(*)
Same query, same data, but no cache hit	Query text differs. Case, alias or whitespace
Cache hit stopped after weeks of daily use	The 31-day ceiling from first execution
Cache lost after a maintenance job, with no DML	Reclustering changed the micro-partitions
First queries slow after every idle period	Warehouse cache dropped on suspend
Benchmarking shows impossibly fast repeat runs	Set USE_CACHED_RESULT = FALSE for the session

ON THE EXAM

Six observations. Four of them are the result cache, which tells you where to spend revision time.

A dashboard runs SELECT region, SUM(amount) FROM sales GROUP BY region every ten minutes and always hits the result cache. An analyst copies the query into a worksheet, adds a table alias and reformats the keywords in lowercase, then runs it.

It takes eleven seconds and consumes warehouse credits. The data has not changed and the analyst holds the same role.

Why was the cached result not reused?

A. Result cache reuse requires the same session. A new worksheet always starts a new session.

B. Aggregations are never cached, because GROUP BY results are recomputed each time.

C. The query text must match exactly. A table alias or a change of case prevents reuse.

D. The dashboard tool holds an exclusive lock on the cached result while it is scheduled.

◆ PAUSE AND DECIDE

Why was the cached result not reused?

A Reuse requires the same session.

The cache is in cloud services and is account-wide. A different session, user or warehouse can reuse it.

B Aggregations are never cached.

Invented. Aggregations cache like anything else, and they are the queries that benefit most.

C The text must match exactly.

Documented, naming case and table aliases specifically. The analyst made two changes, either of which alone would suffice.

D An exclusive lock.

No such mechanism. Cached results are read-only and shareable.

THE SIGNAL

The scenario removes every other explanation: same data, same role. Only the text changed, and that was enough.

> An operations dashboard refreshes every five minutes with an identical query. The underlying table is loaded continuously by Snowpipe, roughly once a minute.
>
> A team lead expects most refreshes to be free from the result cache and is surprised to find that essentially every refresh consumes warehouse credits.

Why is the cache almost never hit?

A Any DML on a referenced table invalidates the cached result, and Snowpipe is loading every minute.

B The result cache only applies to queries run manually, not to queries issued by a BI tool.

C Five minutes is shorter than the minimum cache retention, so results have not persisted yet.

D Snowpipe writes bypass the cache invalidation mechanism, so the cache is serving stale results instead.

◆ PAUSE AND DECIDE

Why is the cache almost never hit?

A DML invalidates it, and Snowpipe loads every minute.

The data must be unchanged for reuse. A load a minute means almost every five-minute refresh sees changed data.

B Only manual queries are cached.

Invented. The cache does not know or care which client submitted the query.

C Five minutes is below a minimum retention.

There is no minimum. A result is available immediately and lasts 24 hours from last use.

D Snowpipe bypasses invalidation.

Would mean stale results, which is the opposite of the symptom. Invalidation is not optional.

THE SIGNAL

The result cache and continuous ingestion pull against each other. Fresh data is exactly what stops a result being reusable.

> An analytics team reports that the first two or three queries after any quiet period take four to five times longer than usual. Then everything is fast again. The queries are varied, so the result cache does not apply.
>
> The warehouse has AUTO_SUSPEND set to 60 seconds. Finance set it that way to eliminate idle credits.

What is happening, and what is the trade?

A. Auto-resume takes several minutes, so the first queries wait for the warehouse to become available.

B. The metadata cache is rebuilt on resume, which is what the first queries are waiting for.

C. The 60-second minimum billing period throttles the first queries after a resume.

D. Suspending drops the warehouse local disk cache. A longer auto-suspend preserves it, at the cost of idle credits.

◆ PAUSE AND DECIDE

What is happening, and what is the trade?

A Auto-resume takes minutes.

Resume is typically a second or two. It would also affect only the first query, not the first two or three.

B The metadata cache is rebuilt.

The metadata cache is in cloud services and survives suspension entirely. Only the warehouse cache is lost.

C The 60-second minimum throttles queries.

That is a billing rule, not a performance one. It affects what you pay, never how fast a query runs.

D The local disk cache is dropped. Longer suspend preserves it.

Documented. The next queries re-read from storage until the cache warms again.

THE SIGNAL

A 60-second auto-suspend can cost more than it saves: repeated 60-second minimums plus a permanently cold cache.

> A monthly board report runs the same query on the first working day of each month. For over a year it returned instantly at no compute cost. This month it took ninety seconds and consumed credits.
>
> The query text is unchanged, the source table is append-only and was last written to eight months ago, and the role is the same.

What is the most likely explanation?

A USE_CACHED_RESULT must have been set to FALSE at account level.

B The result cache is capped at 31 days from first execution, so a monthly query can never reuse a result.

C Monthly queries fall outside the 24-hour window, so this has always been recomputed and the timing was coincidental.

D Fail-safe expired on the source table, so the cached result could no longer be validated.

◆ PAUSE AND DECIDE

What is the most likely explanation?

A USE_CACHED_RESULT set to FALSE.

Possible, and it would have affected every query in the account, not one monthly report. Nothing suggests a global change.

B The 31-day ceiling makes monthly reuse impossible.

24 hours from last use, capped at 31 days from first execution. A gap of a month exceeds both, so a monthly query always regenerates.

C It has always been recomputed. Coincidence.

Contradicts the scenario, which says it returned instantly at no compute cost for over a year.

D Fail-safe expired.

Fail-safe is disaster recovery for dropped data. It has no connection to result cache validity.

THE SIGNAL

Both limits fail here. 24 hours from last use is exceeded, and so is 31 days from first execution.

What to carry into the next module

- **Result and metadata caches are in cloud services.** No warehouse needed, and they survive suspension.
- **The warehouse local disk cache dies with the warehouse.** That is the other half of the auto-suspend trade.
- **24 hours from last use, 31 days from first execution.** Learn the ceiling, not just the window.
- **Exact text, unchanged data, unchanged partitions, right privileges, no volatile functions.** Five ways to miss.
- **Queued, scanning, spilling, exploding join.** Four questions, four remedies.

> ON THE EXAM
>
> Next: reading a Query Profile. The module that turns this decision tree into something you can actually do.